

Hadoop MapReduce Weather Average - Complete Guide

1. Dataset (sample_weather.txt)

```
nano sample_weather.txt
```

Paste:

```
2024-01-01,25,18,12
2024-01-02,28,20,10
2024-01-03,30,21,15
2024-01-04,27,19,11
```

2. Java Code (WeatherAvg.java)

```
nano WeatherAvg.java
```

<PASTE FULL JAVA CODE>

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WeatherAvg {

    public static class Map extends Mapper<LongWritable, Text, Text, FloatWritable> {

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String[] parts = value.toString().split(",");

            if (parts.length == 4) {
                context.write(new Text("Temperature"), new FloatWritable(Float.parseFloat(parts[1])))
                context.write(new Text("DewPoint"), new FloatWritable(Float.parseFloat(parts[2])))
                context.write(new Text("WindSpeed"), new FloatWritable(Float.parseFloat(parts[3])))
            }
        }
    }

    public static class Reduce extends Reducer<Text, FloatWritable, Text, FloatWritable> {

        public void reduce(Text key, Iterable<FloatWritable> values, Context context)
            throws IOException, InterruptedException {

            float sum = 0;
            int count = 0;

            for (FloatWritable val : values) {
                sum += val.get();
                count++;
            }

            context.write(key, new FloatWritable(sum / count));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Weather Average");

        job.setJarByClass(WeatherAvg.class);
        job.setMapperClass(Map.class);
    }
}
```

```

        job.setReducerClass(Reduce.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(FloatWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

3. Commands Sequence

```

start-dfs.sh
start-yarn.sh
jps
hdfs dfs -mkdir /weather
hdfs dfs -put sample_weather.txt /weather/
hdfs dfs -ls /weather
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath ${HADOOP_CLASSPATH} -d . WeatherAvg.java
jar -cvf weather.jar *
hadoop jar weather.jar WeatherAvg /weather/sample_weather.txt /weather/output
hdfs dfs -cat /weather/output/part-r-00000

```

Expected Output

```

DewPoint      19.5
Temperature    27.5
WindSpeed     12.0

```